

```

print("Array containing", arr.nItems, "items")
arr.traverse()

print("Search for 12 returns", arr.search(12))

print("Search for 12.34 returns", arr.search(12.34))

print("Deleting 0 returns", arr.delete(0))
print("Deleting 17 returns", arr.delete(17))

print("Array after deletions has", arr.nItems, "items")
arr.traverse()

```

To run the program, you can use a command-line interpreter to navigate to a folder where both files are present and run the following:

```

$ python3 BadArrayClient.py
Array containing 10 items
77
99
foo
bar
44
55
12.34
0
baz
-17
Search for 12 returns None
Search for 12.34 returns 12.34
Traceback (most recent call last):
  File "BadArrayClient.py", line 23, in <module>
    print("Deleting 0 returns", arr.delete(0))
  File "/Users/canning/chapters/02 code/BadArray.py", line 24, in
delete
    self.__a[k] = self.__a[k+1]    # right over 1
IndexError: list index out of range

```

The results show that most of the methods work properly; this example illustrates the use of the public instance attribute, `nItems`, to provide the number of items in the `Array`. The Python traceback shows that there's a problem with the `delete()` method. The error is that the list index was out of range. That means either `k` or `k+1` was out of range in the line displayed in the traceback. Going back to the code in `BadArray.py`, you can see that `k` lies in

the range of `j` up to but not including `self.nItems`. The index `j` can't be out of bounds because the method already accessed `__a[j]` and found that it matched `item` in the line preceding the `k` loop. When `k` gets to be `self.nItems - 1`, then `k + 1` is `self.nItems`, and that *is outside of the bounds* of the maximum size of the `list` initially allocated. So, we need to adjust the range that `k` takes in the loop to move array items. Before fixing that, let's look more at the details of all the algorithms used.

Insertion

Inserting an item into the `Array` is easy; we already know the position where the insertion should go because we have the number of current items that are stored. [Listing 2-1](#) shows that the item is placed in the internal `list` at the `self.nItems` position. Afterward, the number of items attribute is increased so that subsequent operations will know that that element is now filled. Note that the method does not check whether the allocated space for the `list` is enough to accommodate the new item.

Searching

The `item` variable holds the value being sought. The `search` method steps through only those indices of the internal `list` within the current number of items, comparing the `item` argument with each array item. If the loop variable `j` passes the last occupied element with no match being found, the value isn't in the `Array`. Appropriate messages are displayed by the `BadArrayClient.py` program: Search for 12 returns `None` or Search for 12.34 returns 12.34.

Deletion

Deletion begins with a search for the specified item. If found, all the items with higher index values are moved down one element to fill in the hole in the `list` left by the deleted item. The method decrements the instance's `nItems` attribute, but you've already seen an error happen before that point. Another possibility to consider is what happens if the item isn't found. In the implementation of [Listing 2-1](#), it returns `False`. Another approach would be to raise an exception in that case.

Traversal

Traversing all the items is straightforward: We step through the `Array`, accessing each one via the private instance variable `__a[j]` and apply the `(print)` function to it.

Observations

In addition to the bug discovered, the `BadArray.py` module does not provide methods for accessing or changing arbitrary items in the `Array`. That's a fundamental operation of arrays, so we need to include that. We will keep the code simple and focus attention on operations that will be common across many data structures.

The `Array` class demonstrates **encapsulation** (another aspect of object-oriented programming) by providing the four methods and keeping the underlying data stored in the `__a` list as private. Programs that use `Array` objects are able to access the data only through those methods. The `nItems` attribute is public, which makes it convenient for access by `Array` users. Being public, however, opens that attribute to being manipulated by `Array` users, which could cause errors to occur. We address these issues in the next version of the program.

A Better Array Class Implementation

The next sample program shows an improved interface for the array storage structure class. The class is still called `Array`, and it's in a file called `Array.py`, as shown in [Listing 2-3](#).

The constructor is almost the same except that the variable holding the number of items in the `Array` is renamed `__nItems`. The double underscore prefix marks this instance attribute as private. To make it easy to get the current number of items in `Array` instances, this example introduces a new method named `__len__()`. This is a special name to Python because objects that implement a `__len__()` method can be passed as arguments to the built-in Python `len()` function, like all other sequence types. By calling this function, programs that use the `Array` class can get the value stored in `__nItems` but are not allowed to set its value.

Listing 2-3 *The `Array.py` Module*

```
# Implement an Array data structure as a simplified type of list.
```

```

class Array(object):
    def __init__(self, initialSize):    # Constructor
        self.__a = [None] * initialSize # The array stored as a list
        self.__nItems = 0             # No items in array initially

    def __len__(self):                  # Special def for len() func
        return self.__nItems           # Return number of items

    def get(self, n):                   # Return the value at index n
        if 0 <= n and n < self.__nItems: # Check if n is in bounds, and
            return self.__a[n]           # only return item if in bounds

    def set(self, n, value):            # Set the value at index n
        if 0 <= n and n < self.__nItems: # Check if n is in bounds, and
            self.__a[n] = value          # only set item if in bounds

    def insert(self, item):             # Insert item at end
        self.__a[self.__nItems] = item  # Item goes at current end
        self.__nItems += 1              # Increment number of items

    def find(self, item):               # Find index for item
        for j in range(self.__nItems):  # Among current items
            if self.__a[j] == item:      # If found,
                return j                 # then return index to item
        return -1                       # Not found -> return -1

    def search(self, item):             # Search for item
        return self.get(self.find(item)) # and return item if found

    def delete(self, item):             # Delete first occurrence
        for j in range(self.__nItems):  # of an item
            if self.__a[j] == item:      # Found item
                self.__nItems -= 1       # One fewer at end
                for k in range(j, self.__nItems): # Move items from
                    self.__a[k] = self.__a[k+1]   # right over 1
                return True              # Return success flag

        return False                    # Made it here, so couldn't find the item

    def traverse(self, function=print):  # Traverse all items
        for j in range(self.__nItems):  # and apply a function
            function(self.__a[j])

```

It's important to know about Python's mechanisms to manage "private" attributes. The underscore prefix in a name indicates that the attribute *should*

be treated as private but does not guarantee it. Using a double underscore prefix in an attribute like `__nItems` causes Python to use **name mangling**, making it harder but not impossible to access the attribute. It also makes accessing that attribute in subclasses more complex. To make private attributes that can be easily accessed by the same name in subclasses, use a single underscore prefix. For this example, we choose to keep the double underscore name to illustrate how to control public access, such as using the `__len__()` method.

The example in [Listing 2-3](#) also introduces `get()` and `set()` methods that allow `Array` users to read and write the values of individual elements based on an index. This is the basic functionality of arrays in all languages. The `get()` method checks that the desired index is within the current bounds and returns the value if it is. Note that if the index is out of bounds, there is no explicit return value. Python functions and methods return `None` if execution reaches the end of the function body. The `set()` method checks that the index is in bounds and sets that cell's value, if so, returning `None`.

The `insert()` method remains the same, but we change the `search()` method by breaking it up into two methods. We define a new `find()` method that finds the index to the item being sought. This method loops through the current items and returns the index of the item if it's found, or `-1` if it isn't. We choose `-1` for the return value because it cannot possibly be confused with a valid index value into the `list` and to guarantee the output of `find()` is always an integer (not `None`). The output of the `find()` method can thus be passed to the `get()` method to get the item after finding its index. If the item is not found, `find()` returns `-1`, and `get()` and `search()` still return `None`.

We alter the `delete()` method to fix the index out-of-bounds error in `BadArray.py` by moving the decrement of `__nItems` to occur *before* the loop that moves items to the left in the `list` (see [Listing 2-3](#)). The `traverse()` method remains the same.

Listing 2-4 *The `ArrayClient.py` Program*

```
import Array
maxSize = 10          # Max size of the array
arr = Array.Array(maxSize)    # Create an array object

arr.insert(77)        # Insert 10 items
```

```

arr.insert(99)
arr.insert("foo")
arr.insert("bar")
arr.insert(44)
arr.insert(55)
arr.insert(12.34)
arr.insert(0)
arr.insert("baz")
arr.insert(-17)

print("Array containing", len(arr), "items")
arr.traverse()

print("Search for 12 returns", arr.search(12))

print("Search for 12.34 returns", arr.search(12.34))

print("Deleting 0 returns", arr.delete(0))
print("Deleting 17 returns", arr.delete(17))

print("Setting item at index 3 to 33")
arr.set(3, 33)

print("Array after deletions has", len(arr), "items")
arr.traverse()

```

A new client program, `ArrayClient.py`, exercises the `Array` class, as shown in [Listing 2-4](#). This program is almost identical to `BadArrayClient.py` but uses the new module name, `Array`, and tests the new features of the interface by calling the `len()` function on the `Array` and the `set()` method. The tests that call `search()` are also testing the new `find()` and `get()` methods.

You can confirm that the bug is fixed and no new bugs have shown up by running `ArrayClient.py` to see the following:

```

$ python3 ArrayClient.py
Array containing 10 items
77
99
foo
bar
44
55
12.34
0
baz

```

```
-17
Search for 12 returns None
Search for 12.34 returns 12.34
Deleting 0 returns True
Deleting 17 returns False
Setting item at index 3 to 33
Array after deletions has 9 items
77
99
foo
33
44
55
12.34
baz
-17
```

We now have a functional `Array` class that implements the four core methods for a data storage object. The code shown in the Array visualization tool is that of the `Array` class in [Listing 2-3](#). Try using the visualization tool to search for an item and follow the highlights in the code. You will see that it calls the `search()` method, which calls the `find()` method. Those both show up separated by a gray line. When the `find()` method finishes, its local variables are erased, and its source code disappears, leaving the `search()` method to use its result and try to get the item. The visualization does not show the execution of the `get()` method but does show a message indicating whether the item was found or not.

You can also try out the allocation of a new array. The “New” operation allocates an array of a size you provide (if the cells fit on the screen). If you ask for a large number of cells, it makes them smaller, and the numbers may be hidden. The code shown for the New operation is that of the `__init__()` constructor for the `Array` class. The Random Fill operation fills any empty cells of the current array with random keys. The Delete Rightmost removes the last item from the array. These aren’t methods in the basic `Array` class, but they are helpful for the visualization.

The Ordered Array Visualization Tool

Imagine an array in which the data items are arranged in order of ascending values—that is, with the smallest value at index 0, and each cell holding a value larger than the cell below. Such an array is called an **ordered array**.

When you insert an item into this array, the correct location must be found for the insertion: just above a smaller value and just below a larger one. Then all the larger values must be moved up to make room.

Why would you want to arrange data in order? One advantage is that you can speed up search times dramatically using a **binary search**. At the same time, you are making the insert operation more complex because it must find the proper location for each new item.

To get a feel for what these changes bring, start the Ordered Array Visualization tool, using the procedure described in [Appendix A](#). You see an array; it's similar to the one in the Array Visualization tool, but the data is ordered. [Figure 2-4](#) shows what this tool looks like when it starts.

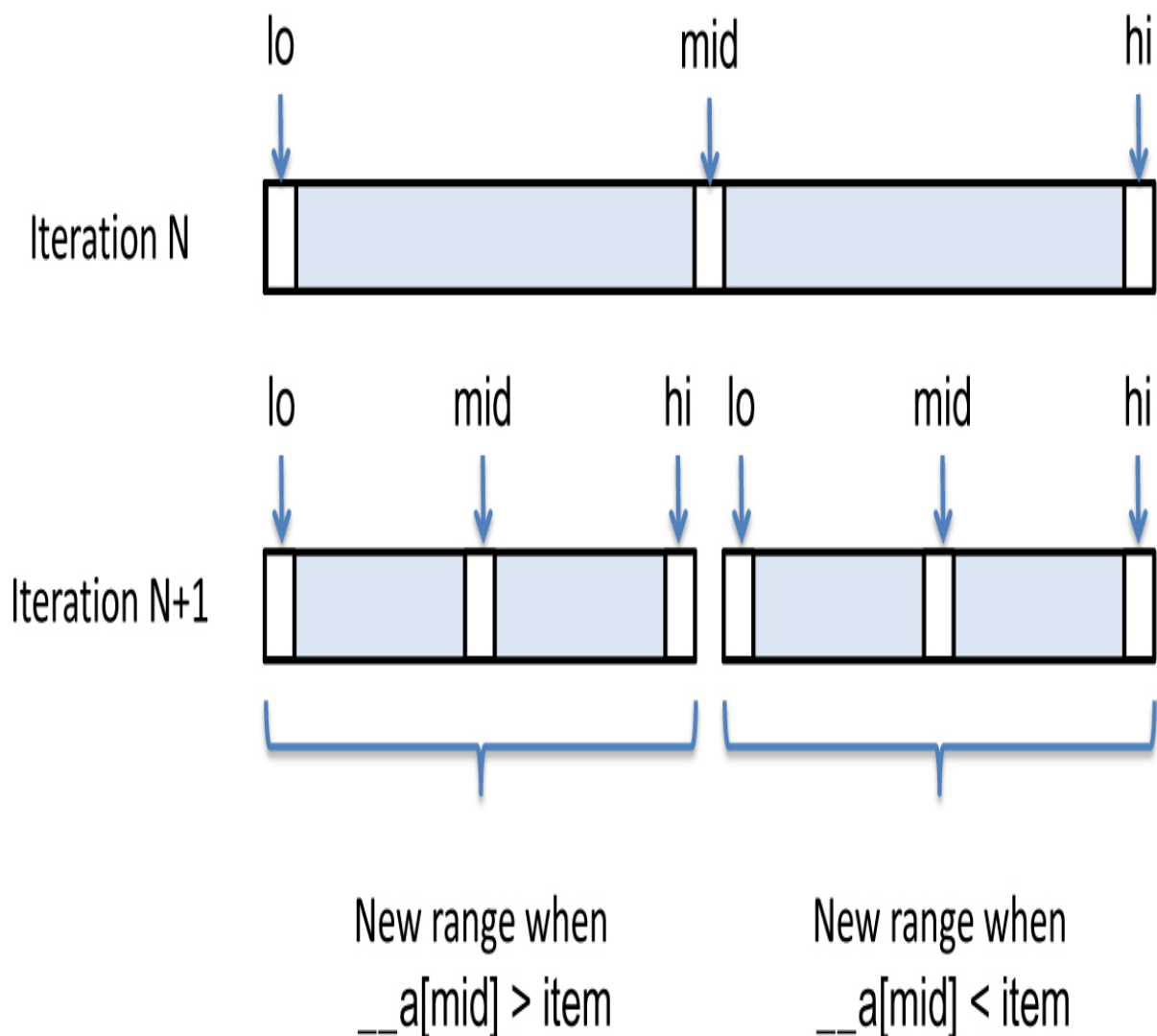


Figure 2-4 *The Ordered Array Visualization tool*

Linear Search

Before we describe how ordering the array helps, we need to elaborate on the kinds of searches we're discussing. The search algorithm used in the (unordered) Array Visualization tool is called a **linear search**. A linear search operates just like someone running a finger over a list of items to find a match. In the visualization, a brown arrow steps along, until it finds a match or reaches the `nItems` limit.

Binary Search

The payoff for using an ordered array comes when you use a binary search. You use this for the Search operation because it is much faster than a linear search, especially for large arrays.

The Guess-a-Number Game

Binary search is a classic approach to guessing games. In the Guess-a-Number game, a friend asks you to guess a number between 1 and 100 ([Figure 2-5](#)). When you guess a number, she tells you one of three things:

- Your guess is larger than the number she's thinking of, or
- It's smaller, or
- You guessed correctly.

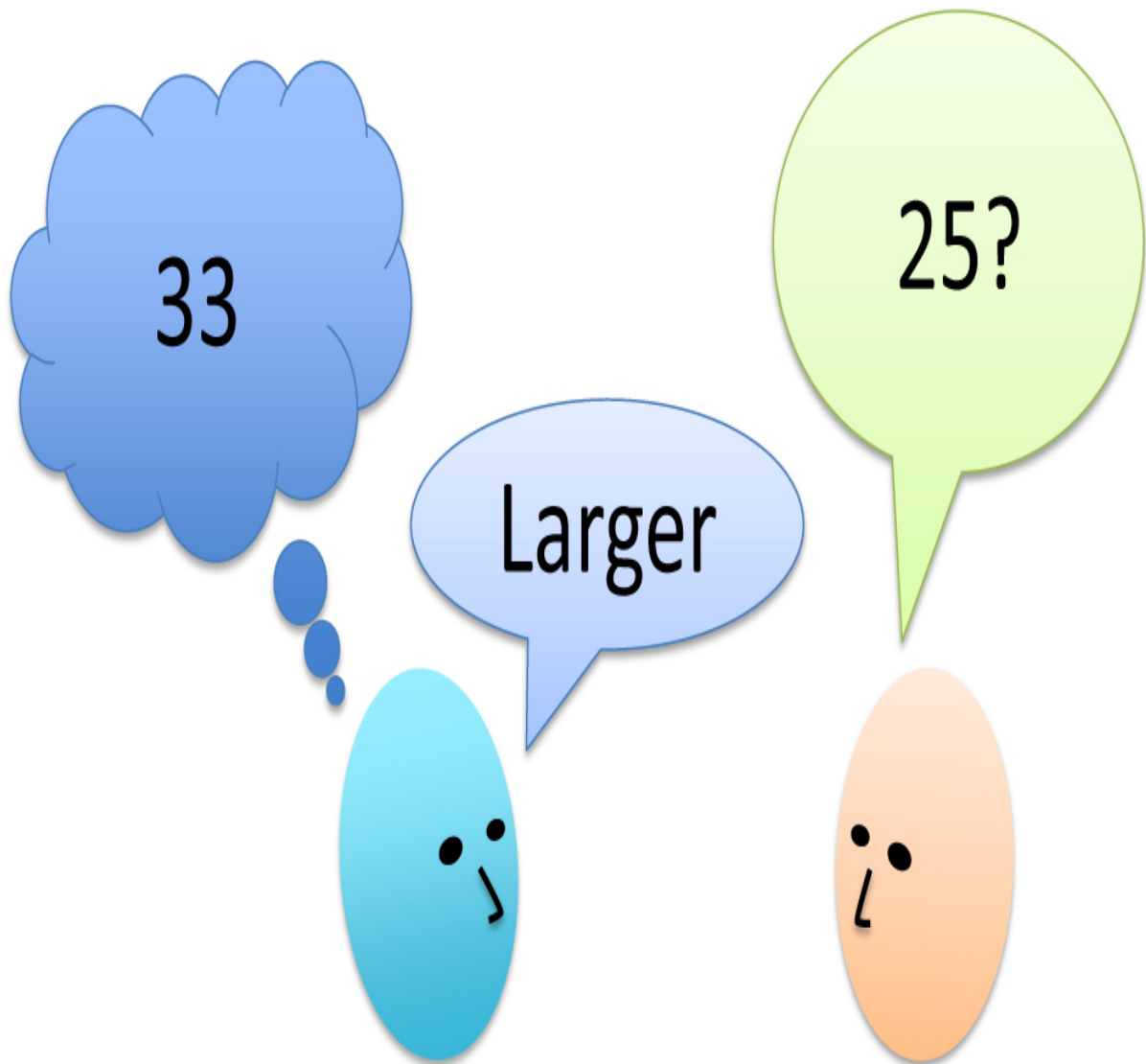


Figure 2-5 *The Guess-a-Number Game*

To find the number in the fewest guesses, you should always start by guessing 50. If your friend says your guess is too low, you deduce the number is between 51 and 100, so your next guess should be 75 (halfway between 51 and 100). If she says it's too high, you deduce the number is between 1 and 49, so your next guess should be 25.

Each guess allows you to divide the range of possible values in half. Finally, if you haven't already found it on an early guess, the range is only one number long, and that's the answer.

Notice how few guesses are required to find the number. If you used a linear search, guessing first 1, then 2, then 3, and so on, finding the number would

take you, on the average, 50 guesses. In a binary search, each guess divides the range of possible values in half, so the number of guesses required is far fewer. [Table 2-2](#) shows a game session when the number to be guessed is 33.

Table 2-2 *Guessing a Number*

Step Number	Number Guessed	Result	Range of Possible Values
0			1–100
1	50	Too high	1–49
2	25	Too low	26–49
3	37	Too high	26–36
4	31	Too low	32–36
5	34	Too high	32–33
6	32	Too low	33–33
7	33	Correct	

The correct number is identified in only 7 guesses. This is the maximum number of guesses, regardless of the number chosen by your friend. You might get lucky and guess the number before you’ve worked your way all the way down to a range of one. This would happen if the number to be guessed was 50, for example, or 34. The most important thing to remember is that you will always find the number in 7 or fewer guesses, which compares well with the maximum of 100 guesses and average of 50 guesses if you search linearly, ignoring the too high and too low clues.

Binary Search in the Ordered Visualization Tool

If you change the Guess-a-Number game into a Where-is-a-Number game, you can use the same strategy in searching the ordered array. It’s a subtle shift, but the question is now asking, “What is the index of the cell holding number X?” The indices range from 0 to $N-1$, so there is the same kind of range to be searched. You can start with the middle array cell and then narrow the range based on what you find there.

Try searching for the newly inserted 55 in ordered array by typing 55 in the text entry box and selecting Search. The Ordered Array Visualization tool shows the array and adds three arrows labeled `lo`, `mid`, and `hi`. The `lo` and `hi` arrows point to the first and last cells of the array, respectively. The `mid` arrow is placed at the midpoint between them, as shown in the top part in [Figure 2-6](#).



Figure 2-6 *Initial ranges in a binary search*

When the range of `lo` to `hi` spans an odd number of cells, the midpoint is the same number of cells from either end, but when it's even, it must be closer to one or the other end. The visualization tool always chooses `mid` to be closer to `lo`, and you'll see why when looking at the code.

After comparing the value at `mid` (59) with the value you're trying to find, the algorithm determines that 55 must lie in the range to the left of `mid`. It moves the `hi` arrow to be one less than `mid` to narrow the range and leaves `lo` unchanged. Then it updates `mid` to be at the midpoint of the narrowed range. The bottom of [Figure 2-6](#) shows this second step of the process.

Each step reduces the range by about half. With the initial 10-element array, the ranges go from 10 to 5, to 2, to 1 at the very most (in [Figure 2-6](#), it goes from 10 to 4 to 2, before finding 55 at index 2). If `mid` happens to point at the goal item, the search can stop. Otherwise, it will continue until the range collapses to nothing.

Try a few searches to see how quickly the visualization tool finds values. Try a search for a value not in the array to see what happens. With a 10-element array, it will take at most four values for `mid` to determine whether the value is present in the array.

What about larger arrays? Use the New operation to find out. Select the text entry box, enter 35, and then select New. If there's enough room in the tool window, it will draw 35 empty cells of a new array. Fill them with random values by selecting Random Fill. The cells show colored rectangles, but the numbers disappear when the cells are too skinny. You can try a variation of the Guess-a-Number game here by typing in a value, selecting Search, and seeing whether it ended up in the array. If you succeed, the tool will add an oval to the cell and provide a success message at the end. You can also select a colored rectangle with your pointer, and it will fill in its value in the text entry area.

Can you figure out how many steps the binary search algorithm will take to find a number based on the size of the array you're searching? We return to this question in the last section of this chapter.

Duplicates in Ordered Arrays

We saw that the presence of duplicate values affected the number of comparisons and shifts needed in searches and deletes on unordered arrays. Does that change for ordered arrays? Yes, a little bit.

Let's look at searching first because it affects insertion and deletion. If finding only a single matching item is sufficient, then there's no difference whether the array has duplicates or not. When you find the first one, you're done. If searches must return *all* matching items, the binary search algorithm finds only the first of them. After that, you would need to find any duplicates to the left or right of the one discovered by binary search. That could be done with linear searching, and it would need to search only the last `lo` to `hi` range explored by the binary search. That would add extra steps, possibly up to visiting all `N` items because the entire array could be duplicates of the same value. On average, however, it would be much less.

Note that a successful binary search does not guarantee finding the item with the lowest or highest index among those with duplicate keys. It guarantees finding one of them, but finding the relative position of that item to the others requires the extra linear searching.

Insertion remains the same for ordered arrays when duplicates are permitted. If a duplicate key exists, the binary search will find one of the duplicates and insert the new item beside it. If the new item has a unique key, it will be placed in order as before. You still need to shift values over to correctly insert any new value. The presence of duplicates might mean shifting fewer values, if the item

to insert matches the value of one or more existing items. The presence of a few duplicates, however, does not significantly change the average number of comparisons and shifts needed, which remain about $N/2$.

For deletions, the effect of duplicates is also a bit complicated. If deleting only one of the matching items is sufficient, you can use binary search to find the item and shift items to the right of it to fill the hole caused by its removal. If there are many duplicates, you could save some shifts by shifting only the rightmost duplicate to fill the hole, but finding the rightmost takes almost as much time as shifting all the duplicates in between.

If deletion requires deleting *all* matching items, the complexity that we discussed for the search operation applies to the deletion process too. There would be fewer shifts needed after finding all the duplicates because you can shift values over D cells as fast as shifting over 1 cell, where D is the number of matching duplicate items. Overall, however, there is not much of a difference compared to the no-duplicates case because the number of operations is still proportional to N .

Python Code for an Ordered Array Class

Let's examine some Python code that implements an ordered array. This example uses the `OrderedArray` class to encapsulate the underlying `list` and its algorithms. The heart of this class is the `find()` method, which uses a binary search to locate a specified data item. We examine this method in detail before showing the complete program.

Binary Search with the `find()` Method

The `find()` method searches for the index to a specified item by repeatedly dividing in half the range of list items to be considered. The method looks like this:

```
def find(self, item):          # Find index at or just below
    lo = 0                    # item in ordered list
    hi = self.__nItems-1      # Look between lo and hi

    while lo <= hi:
        mid = (lo + hi) // 2    # Select the midpoint
        if self.__a[mid] == item: # Did we find it at midpoint?
            return mid          # Return location of item
```

```

elif self.__a[mid] < item: # Is item in upper half?
    lo = mid + 1          # Yes, raise the lo boundary
else:
    hi = mid - 1          # No, but could be in lower half

    return lo            # Item not found, return insertion point instead

```

The method begins by setting the `lo` and `hi` variables to the first and last indices in the array. Setting these variables specifies the range for where the `item` may be found. Then, within the `while` loop, the index, `mid`, is set to the middle of this range.

If you're lucky, `mid` may already be pointing to the desired item, so you first check if `self.__a[mid] == item` is true. If it is, you've found the item, and you return with its index, `mid`.

If `mid` does not point to the `item` being sought, then you need to figure out which half of the range it falls in. You check whether the `item` is bigger than the one at the midpoint by testing `self.__a[mid] < item`. If the `item` is bigger, then you can shrink the search range by setting the `lo` boundary to be 1 above the midpoint. Note that setting `lo` to be the same as `mid` would mean including that midpoint item in the remaining search. You don't want to include it because the comparison with the item at `mid` already showed that its value is too low. Finally, if the midpoint item is neither equal to nor less than the `item` being sought, it must be bigger. In this case, you can shrink the search range by setting `hi` to be 1 below the midpoint. [Figure 2-7](#) shows how the range is altered in these two situations.

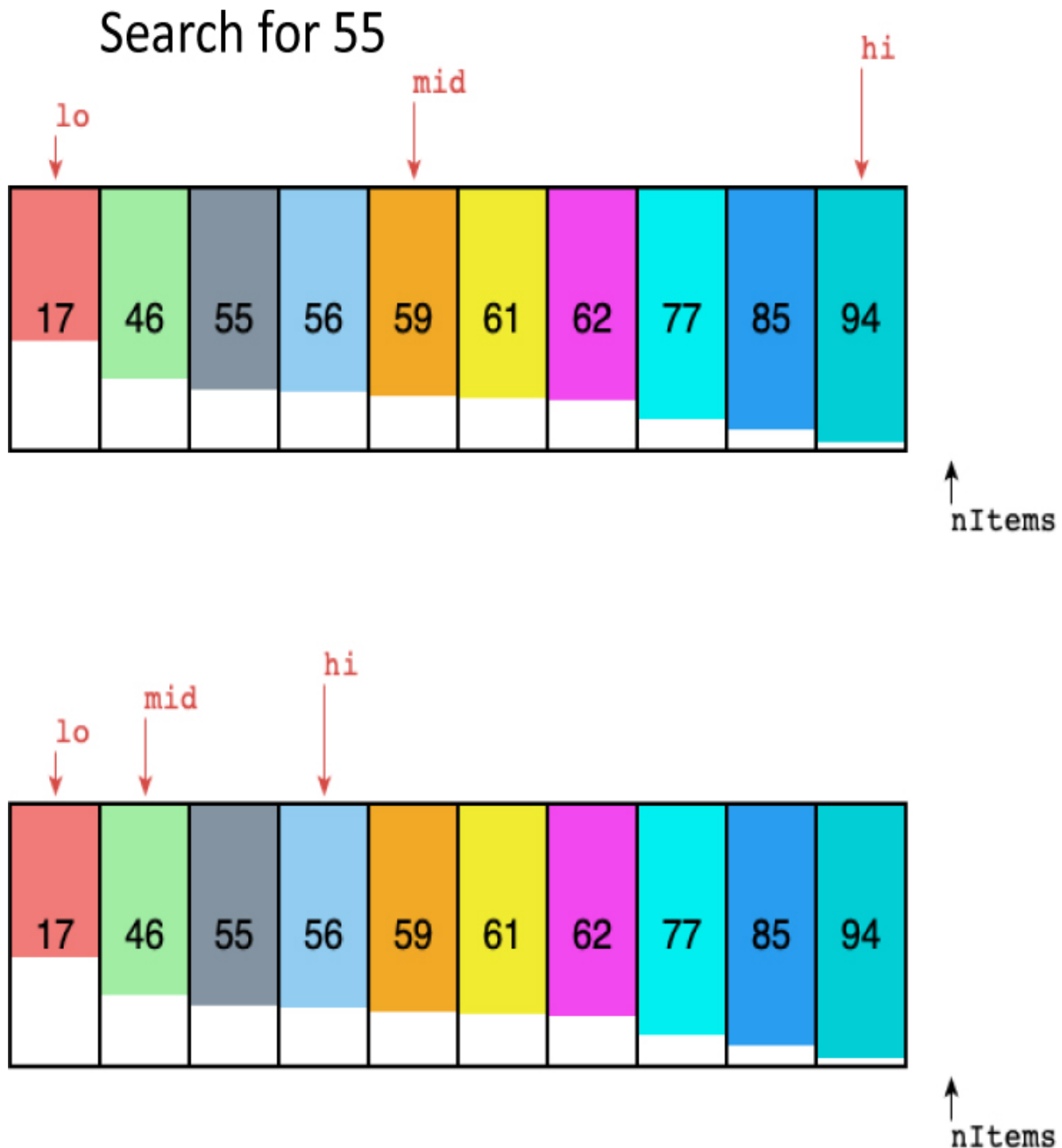


Figure 2-7 *Dividing the range in a binary search*

Each time through the loop you divide the range in half. Eventually, the range becomes so small that it can't be divided any more. You check for this in the loop condition: if `lo` is greater than `hi`, the range has ceased to exist. (When `lo` equals `hi`, the range is one and you need one more pass through the loop.) You can't continue the search without a valid range, but you haven't found the desired item, so you return `lo`, the last lower bound of the search range. This might seem odd because you're returning an index that doesn't point to the

item being sought. It still could be useful, however, because it specifies where an item with that value would be placed in the ordered array.

The OrderedArray Class

In general, the `OrderedArray.py` program is similar to `Array.py` (refer to [Listing 2-3](#)). The main difference is that the `find()` method is changed to do a binary search, as we've discussed. Here, we show the class in two parts.

[Listing 2-5](#) shows the basic class infrastructure including the constructor, utility methods, and the traverse operation. [Listing 2-6](#) shows the other three core operations, including the `find()` method.

Listing 2-5 *The Basic OrderedArray Class Definition*

```
# Implement an Ordered Array data structure

class OrderedArray(object):
    def __init__(self, initialSize):    # Constructor
        self.__a = [None] * initialSize # The array stored as a list
        self.__nItems = 0             # No items in array initially

    def __len__(self):                 # Special def for len() func
        return self.__nItems          # Return number of items

    def get(self, n):                  # Return the value at index n
        if 0 <= n and n < self.__nItems: # Check if n is in bounds, and
            return self.__a[n]           # only return item if in bounds
        raise IndexError("Index " + str(n) + " is out of range")

    def traverse(self, function=print): # Traverse all items
        for j in range(self.__nItems): # and apply a function
            function(self.__a[j])

    def __str__(self):                 # Special def for str() func
        ans = "["                     # Surround with square brackets
        for i in range(self.__nItems): # Loop through items
            if len(ans) > 1:           # Except next to left bracket,
                ans += ", "           # separate items with comma
            ans += str(self.__a[i])    # Add string form of item
        ans += "]"                     # Close with right bracket
        return ans
```

The `OrderedArray` constructor is identical to that of the `Array` class; it allocates a list of the specified `initialSize` and sets the item count to 0. The `__len__()` and `traverse()` methods are identical too. In the `get()` method, one change has been added: it raises an exception if called on an index outside the range of active cells. The `IndexError` is a standard Python exception type used for this condition. A customized string message explains the problem.

The `OrderedArray` class includes a new `__str__()` method in [Listing 2-5](#), which builds a string representation of the data items currently in the array. This is more than just a convenient utility for these tests; a Python object's `__str__()` method is invoked by the built-in `str()` function to create a string in contexts where one is needed, such as when the object is passed to the `print()` function. It uses the same syntax that Python uses for a string version of lists: a list of comma-separated values enclosed in square brackets.

Note that we intentionally leave out the `set()` method because that would allow callers to change values in ways that might not keep the items in order.

Listing 2-6 *The Core Operations of the `OrderedArray` Class*

```
class OrderedArray(object):
...
    def find(self, item):          # Find index at or just below
        lo = 0                    # item in ordered list
        hi = self.__nItems-1     # Look between lo and hi

        while lo <= hi:
            mid = (lo + hi) // 2  # Select the midpoint
            if self.__a[mid] == item: # Did we find it at midpoint?
                return mid        # Return location of item
            elif self.__a[mid] < item: # Is item in upper half?
                lo = mid + 1      # Yes, raise the lo boundary
            else:
                hi = mid - 1      # No, but could be in lower half

        return lo                # Item not found, return insertion point instead

    def search(self, item):
        index = self.find(item)   # Search for item
        if index < self.__nItems and self.__a[index] == item:
            return self.__a[index] # and return item if found

    def insert(self, item): # Insert item into correct position
```

```

        if self.__nItems >= len(self.__a): # If array is full,
            raise Exception("Array overflow") # raise exception

        index = self.find(item)           # Find index where item should go
        for j in range(self.__nItems, index, -1): # Move bigger items
            self.__a[j] = self.__a[j-1]      # to the right

        self.__a[index] = item            # Insert the item
        self.__nItems += 1                # Increment the number of items

    def delete(self, item):                # Delete any occurrence
        j = self.find(item)                # Try to find the item
        if j < self.__nItems and self.__a[j] == item: # If found,
            self.__nItems -= 1              # One fewer at end
            for k in range(j, self.__nItems): # Move bigger items left
                self.__a[k] = self.__a[k+1]
            return True                      # Return success flag

        return False                       # Made it here; item not found

```

Listing 2-6 starts with the `find()` method that implements the binary search algorithm. The `search()` method changes a little from that of the `Array`. It first calls `find()` and verifies that the index returned is in bounds. If not, or if the indexed item doesn't match the sought item, it returns `None`. That means searching for an item not in the array will return `None` without raising an exception.

A check at the beginning of the `insert()` method determines whether the array is full. This is done by comparing the length of the Python list, `__a`, to the number of items currently in the array, `__nItems`. If `__nItems` is equal to (or somehow, larger than) the size of the list, inserting another item will overflow it, so the method raises an exception.

Otherwise, the `insert()` method calls `find()` to locate where the new item goes. Then it uses a loop over the indices to the right of the insertion index to move those items one cell to the right. The loop uses `range(self.__nItems, index, -1)` to go backward through the indices from `__nItems` to `index + 1`. The number of items to be moved could be all `N` of them if the new item is the smallest. On average, it will move half the current items.

The `delete()` method calls `find()` to figure out the location of the item to be deleted and whether it is in the array. If it does find the item, it also must move

half the current items to the left on average. If not, it can return `False` without moving anything.

Like before, we use a separate client program to test the operations of the class and the utility methods. The `OrderedArrayClient.py` program appears in [Listing 2-7](#).

Listing 2-7 *The `OrderedArrayClient.py` Program*

```
from OrderedArray import *

maxSize = 1000 # Max size of the array
arr = OrderedArray(maxSize) # Create the array object

arr.insert(77) # Insert 11 items
arr.insert(99)
arr.insert(44) # Inserts not in order
arr.insert(55)
arr.insert(0)
arr.insert(12)
arr.insert(44)
arr.insert(99)
arr.insert(77)
arr.insert(0)
arr.insert(3)

print("Array containing", len(arr), "items:", arr)

arr.delete(0) # Delete a few items
arr.delete(99)
arr.delete(0) # Duplicate deletes
arr.delete(0)
arr.delete(3)

print("Array after deletions has", len(arr), "items:", arr)

print("find(44) returns", arr.find(44))
print("find(46) returns", arr.find(46))
print("find(77) returns", arr.find(77))
```

Note that you can pass the `arr` variable directly to `print` and expect a reasonable output because of the `__str__()` method. We also use a different form of the `import` statement in `OrderedArrayClient.py`. By importing the

module using the “`from module import *`” syntax, the definitions it contains are added in the same namespace as the client program, not in a new namespace for the module. That means you can create the object using the expression `OrderedArray(maxSize)` instead of `OrderedArray.OrderedArray(maxSize)`. The output of the program looks like this:

```
$ python3 OrderedArrayClient.py
Array containing 11 items: [0, 0, 3, 12, 44, 44, 55, 77, 77, 99, 99]
Array after deletions has 7 items: [12, 44, 44, 55, 77, 77, 99]
find(44) returns 1
find(46) returns 3
find(77) returns 5
```

The last three print statements illustrate some particular cases of the binary search with duplicate entries. The result of `find(46)` shows that even though 46 is not in `arr`, it should be inserted after the first three items to preserve ordering. The `find(44)` finds the first occurrence of 44 at position 1. If `delete(44)` were called at this point, it would delete the first of the 44s currently in the array. By contrast, `find(77)` points at the second of the two 77s in the array. The binary search stops after it finds the first matching item, which could be any instance of an item that appears multiple times.

Advantages of Ordered Arrays

What have we gained by using an ordered array? The major advantage is that search times are much faster than in an unordered array. The disadvantage is that insertion takes longer because all the data items with a higher key value must be moved up to make room. Even though it uses binary search to find where the key belongs, it still must move most of the items in the array.

Deletions are slow in both ordered and unordered arrays because items must be moved down to fill the hole left by the deleted item. There is a bit of speed-up for requests to delete items not in the array. By using `find()`, you can quickly discover whether any items need to be moved in the array. That benefit, however, is reduced when the requested item is in the array. Finding the item with a binary search replaces a linear search over the left side of the array. Both linear and binary searching require shifting items to right of the deleted item.

Going back to insertion for a moment, would it be simpler to skip the binary search of `find()` and just move items to the right until you reach an item

smaller than the item being inserted? That saves a function call to `find()` but requires more comparisons. The binary search algorithm uses way fewer than N comparisons to find the insertion point, and if it doesn't call `find()`, the insert code must compare values for all N items being shifted.

Ordered arrays are therefore useful in situations in which searches are frequent, but insertions and deletions are not. An ordered array might be appropriate for a database of a transport company that tracks the location of its vehicles, for example. The need to add and delete the names of a fleet of vehicles happens much less frequently than the events that require updates to their position, so having fast search find the right vehicle for each position update would be important and worth the increased time needed to add each new vehicle. On the other hand, if the transport company keeps a log of the tasks assigned to each vehicle or their actions in picking up and delivering cargo, there would be frequent additions to the log, but perhaps little need to find a particular log entry. The task log could benefit from the data structure with fast insertion time at the expense of a longer search.

Logarithms

In this section we explain how you can use logarithms to calculate the number of steps necessary in a binary search. If you're a math fan, you can probably skip this section. If thinking about math makes you nervous, give it a try, and make sure to take a long, hard look at [Table 2-3](#).

A binary search provides a significant speed increase over a linear search. In the number-guessing game, with a range from 1 to 100, a maximum of seven guesses is needed to identify any number using a binary search; just as in an array of 100 records, a maximum of seven comparisons is needed to find a record with a specified key value. How about other ranges? [Table 2-3](#) shows some representative ranges and the number of comparisons needed for a binary search.

Table 2-3 *Comparisons Needed in a Binary Search*

Range	Comparisons Needed
10	4
100	7
1,000	10
10,000	14
100,000	17
1,000,000	20
10,000,000	24
100,000,000	27
1,000,000,000	30

Notice the differences between binary search times and linear search times. For very small numbers of items, the difference isn't dramatic. Searching 10 items would take an average of 5 comparisons with a linear search ($N/2$) and a maximum of 4 comparisons with a binary search. But the more items there are, the bigger the difference. With 100 items, there are 50 comparisons in a linear search, but only 7 in a binary search. For 1,000 items, the numbers are 500 versus 10, and for 1,000,000 items, they're 500,000 versus 20. You can conclude that for all but very small arrays, the binary search is greatly superior.

The Equation

You can verify the results of [Table 2-3](#) by repeatedly dividing a range (from the first column) in half until it's too small to divide further. The number of divisions this process requires is the number of comparisons shown in the second column.

Repeatedly dividing the range by two is an algorithmic approach to finding the number of comparisons. You might wonder if you could also find the number using a simple equation. Of course, there is such an equation, and it's worth exploring here because it pops up from time to time in the study of data structures. This formula involves logarithms. (Don't panic yet.)

You have probably already experienced logarithms, without having recognized them. Have you ever heard someone say “a six figure salary” or read about “a deal worth eight figures”? Those simplified expressions tell you the approximate amount of the salary or deal by telling you how many digits are needed to write the number. The number of digits could be found by repeatedly dividing the number by 10. When it’s less than 1, the number of divisions is the number of digits.

The numbers in [Table 2-3](#) leave out some interesting data. They don’t answer such questions as “What is the exact size of the maximum range that can be searched in five steps?” To solve this problem, you can create a similar table, but one that starts at the beginning, with a range of one, and works up from there by multiplying the range by two each time. [Table 2-4](#) shows how this looks for the first seven steps.

Table 2-4 *Powers of Two*

Step s , same as $\log_2(r)$	Range r	Range Expressed as Power of 2 (2^s)
0	1	2^0
1	2	2^1
2	4	2^2
3	8	2^3
4	16	2^4
5	32	2^5
6	64	2^6
7	128	2^7
8	256	2^8
9	512	2^9
10	1,024	2^{10}

For the original problem with a range of 100, you can see that 6 steps don’t produce a range quite big enough (64), whereas 7 steps cover it handily (128).

Thus, the 7 steps that are shown for 100 items in [Table 2-3](#) are correct, as are the 10 steps for a range of 1,000.

Doubling the range each time creates a series that's the same as raising 2 to a power, as shown in the third column of [Table 2-4](#). We can express this power as a formula. If s represents steps (the number of times you multiply by 2—that is, the power to which 2 is raised) and r represents the range, then the equation is

$$r = 2^s$$

If you know s , the number of steps, this tells you r , the range. For example, if s is 6, the range is 2^6 , or 64.

The Opposite of Raising 2 to a Power

The original question was the opposite of the one just described: Given the range, you want to know how many comparisons are required to complete a search. That is, given r , you want an equation that gives you s .

The inverse of raising something to a power is called a **logarithm**. Here's the formula you want, expressed with a logarithm:

$$s = \log_2(r)$$

This equation says that the number of steps (comparisons) is equal to the logarithm to the base 2 of the range. What's a logarithm? The base 2 logarithm of a number r is the number of times you must multiply 2 by itself to get r . In [Table 2-4](#), the step numbers in the first column, s , are equal to $\log_2(r)$.

How do you find the logarithm of a number without doing a lot of dividing? Most calculators and computer languages have a log function. For those that don't, sometimes it can be added as option, such as with Python's `math` module. It might only provide a function for log to the base 10, but you can convert easily to base 2 by multiplying by 3.322. For example, $\log_{10}(100) = 2$, so $\log_2(100) = 2$ times 3.322, or 6.644. Rounded up to the whole number 7, this is what appears in the column to the right of 100 in [Table 2-3](#).

In any case, the point here isn't to calculate logarithms. It's more important to understand the relationship between a number and its logarithm. Look again at [Table 2-3](#), which compares the number of items and the number of steps needed to find a particular item. Every time you multiply the number of items

(the range) by a factor of 10, you add only three or four steps (actually 3.322, before rounding off to whole numbers) to the number needed to find a particular item. This is true because, as a number grows larger, its logarithm doesn't grow nearly as fast. We compare this logarithmic growth rate with that of other mathematical functions when we talk about Big O notation later in this chapter.

Storing Objects

In the examples we've shown so far, we've stored single values in array data structures such as integers, floating-point numbers, and strings. Storing such simple values simplifies the program examples, but it's not representative of how you use data storage structures in the real world. Usually, the data you want to store comprises many values or fields, usually called a **record**. For a personnel record, you might store the family name, given name, birth date, first working date, identification number, and so forth. For a fleet of vehicles, you might store the type of vehicle, the name, the date it entered service, a license tag, and so forth. In object-oriented programs, you want to store the objects themselves in data structures. The objects can represent records.

When storing objects or records in ordered data structures, like the `OrderedArray` class, you need to define the way the records are ordered by specifying a **key** that can be used on all of them. Let's look at how that changes the implementation.

The OrderedRecordArray Class

As shown in the previous examples, you can insert any data type into Python arrays. It's very convenient to allow complex data types to be inserted, deleted, and managed in the arrays, along with the benefits of storing them in order, which makes search faster. To distinguish them (and order them), you need a key for each record. The best way to do that is to define a function that extracts the key from a record and then use that function when comparing keys. By using a function, the array data structure doesn't need to know anything about format or organization of the record. All it must do is pass one of the records, let's say record *R*, as the argument to the function, *F*, to get that record's key, *F(R)*.

The function for the key could be provided to the array data structure in several ways. The program needing the array could define a function with a known name like `array_key` that fetches the key. That approach wouldn't be very portable, and it would make it impossible to have different arrays using different key functions. The key function could also be passed as an argument to the array's methods like `find` and `insert`. That would allow different arrays to use different key functions, but it has a potential problem. If the program using the arrays accidentally passes the wrong key function to an array that ordered its records by a different key, then the records could be out of order using the new key. Instead, it's better to define the key function *when the array is created* and not allow it to change. That's how we implement the `OrderedRecordArray` class, as shown in [Listing 2-8](#).

Listing 2-8 *The Basic OrderedRecordArray Class*

```
# Implement an Ordered Array of Records structure

def identity(x):          # The identity function
    return x

class OrderedRecordArray(object):
    def __init__(self, initialSize, key=identity):    # Constructor
        self.__a = [None] * initialSize    # The array stored as a list
        self.__nItems = 0    # No items in array initially
        self.__key = key    # Key function gets record key

    def __len__(self):    # Special def for len() func
        return self.__nItems    # Return number of items

    def get(self, n):    # Return the value at index n
        if n >= 0 and n < self.__nItems:    # Check if n is in bounds, and
            return self.__a[n]    # only return item if in bounds
        raise IndexError("Index " + str(n) + " is out of range")

    def traverse(self, function=print):    # Traverse all items
        for j in range(self.__nItems):    # and apply a function
            function(self.__a[j])

    def __str__(self):    # Special def for str() func
        ans = "["    # Surround with square brackets
        for i in range(self.__nItems):    # Loop through items
            if len(ans) > 1:    # Except next to left bracket,
```

```
ans += ", " # separate items with comma
ans += str(self.__a[i]) # Add string form of item
ans += "]" # Close with right bracket
return ans
```

The constructor for `OrderedRecordArray` takes a new argument, `key`, which is the key function. That function defaults to being the `identity` function, which is defined in the module and simply returns the first argument as the result.

This makes the default behavior the same as the `OrderedArray` class shown in [Listings 2-5](#) and [2-6](#). The key function is stored in the private instance variable `__key` so it should not be modified by the clients using `OrderedRecordArrays`.

[Listing 2-9](#) shows that the `find()` and `search()` methods change to take a `key` as an argument, instead of the item or record used in the `OrderedArray` class. This key is a value, not a function, and is used to compare with the keys extracted from the records in the array. The `find` and `search` methods use the internal `__key` function on the records to get the right value to compare with the `key` being sought. The `insert` and `delete` method signatures don't change—they still operate on item records—but internally they change the way they pass the appropriate key to `find()`.

Listing 2-9 *The Item Operations of the `OrderedRecordArray` Class*

```
class OrderedRecordArray(object):
...
    def find(self, key): # Find index at or just below key
        lo = 0 # in ordered list
        hi = self.__nItems-1 # Look between lo and hi

        while lo <= hi:
            mid = (lo + hi) // 2 # Select the midpoint

            if self.__key(self.__a[mid]) == key: # Did we find it?
                return mid # Return location of item

            elif self.__key(self.__a[mid]) < key: # Is key in upper half?
                lo = mid + 1 # Yes, raise the lo boundary

        else:
            hi = mid - 1 # No, but could be in lower half

        return lo # Item not found, return insertion point instead
```

```

def search(self, key):
    idx = self.find(key)    # Search for a record by its key
    if idx < self.__nItems and self.__key(self.__a[idx]) == key:
return self.__a[idx]        # and return item if found

def insert(self, item):    # Insert item into the correct position
    if self.__nItems >= len(self.__a): # If array is full,
raise Exception("Array overflow") # raise exception

    j = self.find(self.__key(item))    # Find where item should go

    for k in range(self.__nItems, j, -1): # Move bigger items right
self.__a[k] = self.__a[k-1]

    self.__a[j] = item        # Insert the item
    self.__nItems += 1        # Increment the number of items

def delete(self, item):    # Delete any occurrence
    j = self.find(self.__key(item))    # Try to find the item
    if j < self.__nItems and self.__a[j] == item: # If found,
self.__nItems -= 1        # One fewer at end
    for k in range(j, self.__nItems): # Move bigger items left
        self.__a[k] = self.__a[k+1]
    return True            # Return success flag

    return False            # Made it here; item not found

```

The test program for this new class, `OrderedRecordArrayClient.py` shown in [Listing 2-10](#), uses records with two elements or fields. The key for the records is set to be the second element of each record. Loops in this version perform the insertions, deletions, and searches.

Listing 2-10 *The `OrderedRecordArrayClient.py` Program*

```

from OrderedRecordArray import *

def second(x):    # Key on second element of record
    return x[1]

maxSize = 1000    # Max size of the array
arr = OrderedRecordArray(maxSize, second)    # Create the array object

# Insert 10 items

```

```

for rec in [('a', 3.1), ('b', 7.5), ('c', 6.0), ('d', 3.1),
            ('e', 1.4), ('f', -1.2), ('g', 0.0), ('h', 7.5),
            ('i', 7.5), ('j', 6.0)]:
    arr.insert(rec)

print("Array containing", len(arr), "items:\n", arr)

# Delete a few items, including some duplicates
for rec in [('c', 6.0), ('g', 0.0), ('g', 0.0),
            ('b', 7.5), ('i', 7.5)]:
    print("Deleting", rec, "returns", arr.delete(rec))

print("Array after deletions has", len(arr), "items:\n", arr)

for key in [4.4, 6.0, 7.5]:
    print("find(", key, ") returns", arr.find(key),
          "and get(", arr.find(key), ") returns",
          arr.get(arr.find(key)))

```

After putting 10 records in the array including some with duplicate keys, the test program deletes a few records, showing the result of the deletion. It then tries to find a few keys in the reduced array. The result of running the program is

```

$ python3 OrderedRecordArrayClient.py
Array containing 10 items:
[('f', -1.2), ('g', 0.0), ('e', 1.4), ('d', 3.1), ('a', 3.1), ('j',
6.0), ('c', 6.0), ('i', 7.5), ('h', 7.5), ('b', 7.5)]
Deleting ('c', 6.0) returns False
Deleting ('g', 0.0) returns True
Deleting ('g', 0.0) returns False
Deleting ('b', 7.5) returns False
Deleting ('i', 7.5) returns True
Array after deletions has 8 items:
[('f', -1.2), ('e', 1.4), ('d', 3.1), ('a', 3.1), ('j', 6.0), ('c',
6.0), ('h', 7.5), ('b', 7.5)]
find( 4.4 ) returns 4 and get( 4 ) returns ('j', 6.0)
find( 6.0 ) returns 5 and get( 5 ) returns ('c', 6.0)
find( 7.5 ) returns 6 and get( 6 ) returns ('h', 7.5)

```

The program output shows that deleting the record `('c', 6.0)` fails. Why? The next two deletions show that deleting `('g', 0.0)` succeeds the first time but fails the second time because only one record has that key, 0.0. That's what is expected, but the next deletions are unexpected. The deletion of the record `('b', 7.5)` fails, but the deletion of `('i', 7.5)` succeeds. What is going on?

The issue comes up because of the *duplicate keys*. The program inserts three records that have the key 7.5. When the `find()` method runs, it uses binary search to get the index to one of those records. The exact one it finds depends on the sequence of values for the `mid` variable. You can see which one it finds in the output of the `find` tests. Note that `find(4.4)` returns a valid index, 4, and that points to the location where a record with that key should go. The record at index 4 has the next higher key value, 6.0. When you call `find(7.5)` on the final `Array`, it returns 6, which points to the `('h', 7.5)` record. That isn't equal to the `('b', 7.5)` record using Python's `==` test. The `delete()` method removes only items that pass the `==` test. You can also deduce that `find(7.5)` did find the `('i', 7.5)` record on the earlier delete operation. This example illustrates an important issue when duplicate keys are allowed in a sorted data structure like `OrderedRecordArray`. One of the end-of-chapter programming projects asks you to change the behavior of this class to correctly delete records with duplicate keys.

Big O Notation

Which algorithms are faster than others? Everyone wants their results as soon as possible, so you need to be able compare the different approaches. You can certainly run experiments with each program on a particular computer and with a particular set of data to see which is fastest. That capability is useful, but when the computer changes or the data changes, you could get different results. Computers generally get faster as better technologies are invented, and that makes all algorithms run faster. The changes with the data, however, are harder to predict. You've already seen that a binary search takes far fewer steps than a linear search because the number of items to search increases. We'd like to be able to extend that reasoning to help predict what will happen with other algorithms.

People like to categorize things, especially by what they are capable of doing. If you think about cutting grass, there are push lawn mowers, powered lawn mowers, riding lawn mowers, and towed grass cutters. Each one of them is good for different size jobs of grass cutting. Similarly for refrigeration, there are personal refrigerators, household refrigerators, restaurant kitchen refrigerators, walk-in refrigerators, and refrigerated warehouses for different quantities of perishable items. In each case, choosing something too big or too small for the job would be costly in time or money.

In computer science, a rough measure of performance called **Big O** notation is used to describe algorithms. It's primarily used to describe the speed of algorithms but is also used to describe how much storage they need. Algorithms with the same Big O speed are in the same category. The category gives a rough idea of what amount of data they can process (or storage they need). For example, the linear search `Array` class in [Listing 2-3](#) should be plenty fast for small jobs like keeping a list of contacts in a personal computer or phone or watch. It would probably not be acceptable for the list of contacts of a large corporation with tens of thousands of employees, and it certainly would be too slow to manage all the contact information for a nation of tens of millions of people. By using the `OrderedRecordArray` class in [Listing 2-8](#), you can get the benefit of binary search and drastically reduce the search time by making it proportional to the logarithm of the number of items. Are there even better algorithms? Big O notation helps answer that question.

Insertion in an Unordered Array: Constant

Insertion into an unordered array is the only algorithm we've discussed that doesn't depend on how many items are in the array. The new item is always placed in the next available position, at `__a[self.__nItems]`, and `self.__nItems` is then incremented. Instead of using the variable names in a particular program, Big O notation uses N to stand for the number of items being managed. Insertion into an unordered array requires the same amount of time no matter how big N is. You can say that the time, T , to insert an item into an unsorted array is a constant, K :

$$T = K$$

In a real situation, the actual time required by the insertion is related to the speed of the processor, how efficiently the compiler has generated the program code, how much data is in the item being copied into the array, and other factors. The constant K in the preceding equation is used to account for all such factors. To find out what K is in a real situation, you need to measure how long an insertion took. (Software exists for this very purpose.) K would then be equal to that time.

Linear Search: Proportional to N

You've seen that, in a linear search of items in an array, the number of comparisons that must be made to find a specified item is, on the average, half of the total number of items. Thus, if N is the total number of items, the search time T is proportional to half of N :

$$T = K \times N / 2$$

As with insertions, discovering the value of K in this equation would require timing searches for some (probably large) values of N and then using the resulting values of T to calculate K . There is probably some time spent launching the program and cleaning up when it's done that would add a small constant factor to the total time. By measuring the time taken for several searches, you can account for the variations for where the item falls in the array and for the extra constant factors added by the measurement. When you know K and any additional constants, you can calculate T for any other value of N .

For a handier formula, you could lump the 2 into the K . The new K is equal to the old K divided by 2. Now you have

$$T = K \times N$$

This equation says that average linear search times are proportional to the size of the array. If an array is twice as big, searching it will take twice as long. In this case, we are less concerned with getting a precise estimate of the time it will take as we are knowing how fast it will grow as N gets bigger.

Binary Search: Proportional to $\log(N)$

Similarly, you can concoct a formula relating T and N for a binary search:

$$T = K \times \log_2(N)$$

As you saw earlier, the search time is proportional to the base 2 logarithm of N . Actually, because any logarithm is related to any other logarithm by a constant (for example, multiplying by 3.322 to go from base 2 to base 10), you can lump this constant into K as well. Then you don't need to specify the base:

$$T = K \times \log(N)$$

Don't Need the Constant

Big O notation looks like the formulas just described, but it dispenses with the constant K . When comparing algorithms, you don't really care about the particular processor or compiler; all you want to compare is how T changes for different values of N , not what the actual numbers are. Although the K might be important for getting a precise estimate for small values of N , when N is really large, it “dominates” the time calculation. Therefore, we drop the constant in Big O notation.

Big O notation uses the uppercase letter O , which you can think of as meaning “order of.” In Big O notation, you would say that a linear search takes $O(N)$ or “Order of N ” or simply “Order N ” time, and a binary search takes $O(\log(N))$ time. A further simplification of the notation gets rid of the parentheses for the log function, and you simply write $O(\log N)$. Insertion into an unordered array takes $O(1)$, or constant time. (That's the numeral 1 in the parentheses.)

[Table 2-5](#) summarizes the running times of the algorithms we've discussed so far.

Table 2-5 *Running Times in Big O Notation*

Algorithm	Unordered Array	Ordered Array
Linear search	$O(N)$	
Binary search	Not possible	$O(\log N)$
Insertion	$O(1)$	$O(N)$
Deletion	$O(N)$	$O(N)$

You might ask why deletion in ordered arrays isn't shown as $O(\log N) + O(N)$ or maybe $O(\log N + N)$ because it uses binary search to find the location of the item to delete. The reason is that the $O(N)$ part needed for shifting the items of the array is so much larger than the $O(\log N)$ part that it really doesn't matter when N gets big. Big O notation is intended to describe how the algorithm behaves for very large numbers of items.

[Figure 2-8](#) graphs some Big O relationships between time (in number of steps) and number of items, N . Based on this graph, you might rate the various Big O values (very subjectively) like this:

- $O(1)$ is excellent,

- $O(\log N)$ is good,
- $O(N)$ is fair,
- $O(N \times \log N)$ is poor, and
- $O(N^2)$ is bad.

$O(N \times \log N)$ occurs in many kinds of sorting. $O(N^2)$ occurs in simple sorting and in certain graph algorithms, all of which we look at later in this book.



Figure 2-8 *Graph of Big O times*

The idea in Big O notation isn't to give actual figures for running times but to convey how the running time grows as the number of items increases. This is the most meaningful way to compare algorithms, without actually measuring running times in a real installation. This is often called the **computational complexity** of algorithms, or the **order of the function** that characterizes the running time (that's where the "O" comes from). Algorithms that are more complex for the computer to process have a higher order and are usually avoided.

Why Not Use Arrays for Everything?

Arrays seem to get the job done, so why not use them for all data storage? You've already seen some of their disadvantages. In an unordered array, you can insert items quickly, in $O(1)$ time, but searching takes slow $O(N)$ time. In an ordered array, you can search quickly, in $O(\log N)$ time, but insertion takes

$O(N)$ time. For both kinds of arrays, deletion takes $O(N)$ time because half the items (on the average) must be moved to fill in the hole.

It would be nice if there were data structures that could do everything—insertion, deletion, and searching—quickly, ideally in $O(1)$ time, but if not that, then in $O(\log N)$ time. Traversal, by definition, needs $O(N)$ time, but in more complex data structures, it could be larger. In the chapters ahead, we examine how closely these ideals can be approached and the price that must be paid in complexity.

Another problem with arrays is that their size is fixed when they are first created. The reason for that is the compiler needs to know how much space to set aside for the whole array and keep it separate from all the other data. Usually, when the program first starts, you don't know exactly how many items will be placed in the array later, so you guess how big it should be. If the guess is too large, you'll waste memory by having cells in the array that are never filled. If the guess is too small, you'll overflow the array, causing at best a message to the program's user, and at worst a program crash.

Other data structures are more flexible and can expand to hold the number of items inserted in them. The linked list, discussed in [Chapter 5](#), is such a structure. One of the programming projects in this chapter asks you to make an expanding array data structure.

Summary

- Arrays are sequential groupings of data elements. Each element can store a value called an item.
- Each element of the array can be accessed by knowing the start of the array and an integer index to the element.
- Object-oriented programs are used to implement data structures to encapsulate the algorithms that manipulate the data.
- Data structures use private instance variables to restrict access to important values of the structure that could cause errors if changed by the calling program.
- Unordered arrays offer fast insertion but slow searching and deletion.

- A binary search can be applied to an ordered array.
- The logarithm to the base B of a number A is (roughly) the number of times you can divide A by B before the result is less than 1.
- Linear searches require time proportional to the number of items in an array.
- Binary searches require time proportional to the logarithm of the number of items.
- Data structures usually store complex data types like records.
- A key must be defined to order complex data types.
- If duplicate items or keys are allowed in a data structure, the algorithms should have a predictable behavior for how they are managed.
- Big O notation provides a convenient way to compare the speed of algorithms.
- An algorithm that runs in $O(1)$ time is the best, $O(\log N)$ is good, $O(N)$ is fair, and $O(N^2)$ is bad.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. Aside from the insert, delete, search, and traverse methods common to all “database” data structures, array data structures should have _____ method(s).
2. When constructing a new instance of an `Array` ([Listing 2-3](#)):
 - a. the initial value for at least one of the array cells must be set.
 - b. the data type of all the array cells must be set.
 - c. the key for each data item must be set.
 - d. the maximum number of cells the array can hold must be set.
 - e. none of the above.

3. Why is it important to use private instance attributes like `__nItems` in the definition of the array data structure?
4. Inserting an item into an unordered array
 - a. takes time proportional to the size of the array.
 - b. requires multiple comparisons.
 - c. requires shifting other items to make room.
 - d. takes the same time no matter how many items there are.
5. True or False: When you delete an item from an unordered array, in most cases you shift other items to fill in the gap.
6. In an unordered array, allowing duplicates
 - a. increases times for all operations.
 - b. increases search times in some situations.
 - c. always increases insertion times.
 - d. sometimes decreases insertion times.
7. True or False: In an unordered array, it's generally faster to find out an item is not in the array than to find out it is.
8. Ordered arrays, compared with unordered arrays, are
 - a. much quicker at deletion.
 - b. quicker at insertion.
 - c. quicker to create.
 - d. quicker at searching.
9. Keys are used with arrays
 - a. to provide a single value for each array item that can be used to order the items.
 - b. to decrypt the values stored in the array cell.
 - c. to decrease the insertion time in unordered arrays.
 - d. to allow complex data types to be stored as a single key value in the array.

10. The `OrderedArray.py` (Listing 2-6) and `OrderedRecordArray.py` (Listing 2-9) modules have both a `find()` and a `search()` method. How are the two methods the same and how do they differ?
11. A logarithm is the inverse of _____.
12. The base 10 logarithm of 1,000 is _____.
13. The maximum number of items that must be examined to complete a binary search in an array of 200 items is
 - a. 200.
 - b. 8.
 - c. 1.
 - d. 13.
14. The base 2 logarithm of 64 is _____.
15. True or False: The base 2 logarithm of 100 is 2.
16. Big O notation tells
 - a. how the speed of an algorithm relates to the number of items.
 - b. the running time of an algorithm for a given size data structure.
 - c. the running time of an algorithm for a given number of items.
 - d. how the size of a data structure relates to the speed of one of its algorithms.
17. $O(1)$ means a process operates in _____ time.
18. Advantages of using arrays include
 - a. the variable size of array cells.
 - b. the variable length of the array over the lifetime of the data structure.
 - c. the $O(1)$ access time to read or write an array cell.
 - d. the $O(1)$ time to traverse all the items in the array.
 - e. all of the above.
19. A colleague asks for your comments on a data structure using an unordered array without duplicates and binary search. Which of the following comments makes sense?

- a. Because the array can store any data type, a binary search won't be efficient.
 - b. Because the array is unordered, a binary search cannot guarantee finding the item being sought.
 - c. Because binary search takes $O(N)$ time, it would be better to use an ordered array.
 - d. Because the array doesn't have duplicates, binary search doesn't really have an advantage over the simpler linear search.
20. You've been asked to adapt some code that maintains a record about each planet and their moons in a solar system like ours into a system that will store a record about every planet and moon in every known galaxy. The record structure will be a little larger for each planet to hold some new attributes. It's likely that the records will be added and updated "randomly" as telescopes and other sensors point at different parts of the universe over time, filling in some initial attributes of the records, and then updating others during frequent observations. The current code uses an unordered array for the records. Would you recommend any changes? If so, why?

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

- 2-A Use the Array Visualization tool to insert, search for, and delete items. Make sure you can predict what it's going to do. Do this both with duplicate values present and without.
- 2-B Make sure you can predict in advance what indices the Ordered Array Visualization tool will select at each step for `lo`, `mid`, and `hi` when you search for the lowest, second lowest, one above middle, and highest values in the array.
- 2-C In the Ordered Array Visualization tool, create an array of 12 cells and then use the Random Fill button to fill them with values. Use the Delete Rightmost button to remove the five highest values. Then insert five of the same value, somewhere in the middle of the array. Note the colors that are assigned to inserted values and the order they were inserted.